

BACKEND · NODE + EXPRESS + MONGODB

Toàn bộ luật dự án

Chợ Tốt Clone — bản liệt kê đầy đủ, gom từ mọi tài liệu luật trong source

Nguồn	docs/Vue/
Tài liệu gốc	AGENT.md + 6 file trong docs/rules/
Quy mô	9 phần · 13 quy tắc nền · 10 quy tắc thư mục · 6 bất biến multi-tenant · 8 điều cấm tuyệt đối
Ngày lập	02/10/2026

Bản này **liệt kê**, không diễn giải lại. Mỗi phần ghi rõ file nguồn; khi tài liệu gốc đổi thì tài liệu gốc thắng. Phần 8 nói thật về chuyện luật nào có máy chặn và luật nào chỉ sống nhờ code review — đó là thứ không nằm trong bất kỳ file gốc nào. Phần 9 ghi một mâu thuẫn đã phát hiện giữa hai mục trong cùng một file.

0 Bản đồ tài liệu luật

Khảo sát toàn bộ docs/Vue/

Luật trong repo này nằm ở 7 file, tổng 977 dòng. Không có file nào khác chứa luật ràng buộc.

File	Dòng	Trả lời câu hỏi gì
AGENT.md	108	Luật nền — stack, cấu trúc, 13 quy tắc code, quy trình, giới hạn của AI
docs/rules/multi-tenant.convention.md	330	Cách ly dữ liệu khách hàng. File lớn nhất và nghiêm nhất
docs/rules/xu-wallet.decision.md	131	ADR hệ Xu — ví, sổ cái, gói tin, điều kiện bất phí
docs/rules/folder.convention.md	117	File mới đặt ở đâu, đặt tên thế nào
docs/rules/typesaft.convention.md	114	Luật TypeScript — any , return type, DTO, strict
docs/rules/api-contract.md	98	Hợp đồng API — OpenAPI, version, phân trang, lỗi
docs/rules/logging-monitoring.md	79	Luật ghi log và dữ liệu cấm log

Ba mức độ dùng trong tài liệu này

BẮT BUỘC vi phạm là reject · **CÓ MÁY CHẶN** lint/hook/test bắt được · **REVIEW-ONLY** chỉ người đọc mới phát hiện

1 Luật nền

AGENT.md

1.1 Stack — không tự đổi, không tự thêm

Runtime	Node.js 20+, Express 4.x, TypeScript — không có file .js trong src/
DB	MongoDB + Mongoose 8
Auth	JWT (access + refresh), hash mật khẩu bằng <code>@node-rs/bcrypt</code>
Upload ảnh	<i>chưa có</i> — module <code>upload</code> trả 501, chưa chọn thư viện lẫn nơi lưu
Realtime	Socket.io, adapter in-memory → chạy đúng một instance
Validation	Zod + <code>@asteasolutions/zod-to-openapi</code> — một schema dùng cho cả hai
Logger	winston (<code>src/config/logger.ts</code>)
Test	Vitest + Supertest + mongodb-memory-server
Lint	oxlint + Prettier — KHÔNG phải ESLint, không tự đổi rule

1.2 Bày tầng của một feature

```
src/features/<feature>/
<feature>.routes.ts      ← endpoint + middleware + registerPath cho OpenAPI
<feature>.controller.ts
<feature>.service.ts     ← business logic
<feature>.repository.ts  ← toàn bộ truy vấn Mongo (nếu feature có model)
<feature>.model.ts       ← Mongoose schema
<feature>.schema.ts      ← Zod schema (KHÔNG đặt tên .validator.ts)
<feature>.types.ts       ← DTO + hàm map document → DTO
```

Module mẫu để bám theo: `src/features/listing` (đủ 7 tầng) và `src/features/auth` (mẫu DTO + OpenAPI). **Không có module product** — đừng đi tìm.

1.3 Mười ba quy tắc code **BẮT BUỘC**

- 1 Controller không chứa logic nghiệp vụ.**
Chỉ gọi service và trả response.
- 2 Mọi input từ client phải validate bằng Zod**
qua `validate({ body, query, params })` trước khi vào controller.
- 3 Mọi lỗi throw qua `ApiError`**
hoặc subclass trong `src/common/errors/`. Không throw `string/Error` thường. Handler dùng chung:
`src/middlewares/error.middleware.ts`.
- 4 Response thành công luôn đi qua `success()` / `created()`**
trong `src/common/utils/apiResponse.ts` → `{ success, message, data, meta? }`.
- 5 Phân quyền theo phạm vi, không theo vai trò trong org.**
Route cần đăng nhập → `authenticate`; route public tùy chọn → `optionalAuth`. Middleware ở tầng route cố tình **rộng**
để người có phạm vi hẹp vẫn mở được màn hình của họ; phạm vi thật chốt ở service bằng
`src/common/authz/policy.ts` (hàm thuần). `requireAnyModerator` cố tình KHÔNG đòi `requireOrg`, vì người phụ trách danh mục không thuộc nhóm nào.

6 Không hardcode chuỗi trạng thái.

Dùng const trong `src/common/constants/index.ts`. `MEMBERSHIP_ROLES` =
thân phận
trong org; `SYSTEM_ROLES` =
quyền hạn
, phân biệt phạm vi bằng `SCOPE_TYPES`. Gộp hai thứ vào một cột là lỗi thiết kế v1 đã bỏ.

7 Endpoint public không bao giờ được trả tin ngoài `PUBLIC_LISTING_STATUSES`.

Filter mặc định nằm ở `buildFilter()` — đừng bỏ nó khi thêm query mới.

8 Query chỉ đọc → cân nhắc `.lean()`.

Field dùng để filter/sort phải có index.

9 Không dùng `console.log`.

Dùng `logger`. Chữ ký winston là `logger.info('message', { meta })` —
không phải pino
, đảo thứ tự là mất nội dung log.

10 Soft delete.

Model có hook `pre(/^find/)` loại `deletedAt != null`. Hook đó
không
áp cho `countDocuments` — model nào đếm thì phải đăng ký thêm `pre('countDocuments')`.

11 Không commit `.env` hoặc `secret`.

`.env.example` chỉ chứa placeholder.

12 Multi-tenant

— xem phần 6. Bốn điều không được quên dù không mở tài liệu: (a) collection mới → gán `tenantPlugin`; (b) không tự viết filter `organizationId`; (c) ghi luôn rơi về org của request; (d) chạm dữ liệu ngoài request → bọc `runUnscoped('lý do', ...)`.

13 Index trên collection có tenant lấy `organizationId` làm khoá đầu.

Text index bị
cấm
. Đúng ba ngoại lệ (TTL, collection `dualAxis`, đường đọc trong `runUnscoped`), mỗi cái phải có ghi chú ngay tại chỗ khai index. Nghi ngờ index có được dùng không thì
đo
bằng `npm run explain:indexes`, đừng đoán.

1.4 Quy trình khi viết code

1 Đọc feature tương tự đã có để bám convention —

không tự sáng tạo pattern mới

2 `npm run lint` (tự fix + format).

3 `npm run typecheck`, có lỗi thì fix.

4 `npm test` — phải pass trước khi coi task hoàn thành.

5 Đổi schema MongoDB → note rõ trong PR có cần migration không.

6 Không tự ý cài thêm package

ngoài `package.json` hiện có. Cần thì hỏi trước.

1.5 Bốn việc AI không được tự ý làm

- Không xóa / sửa migration cũ.
- Không đổi cấu trúc response API hiện có (breaking change) nếu không được yêu cầu rõ.
- Không tắt / sửa rule lint để né lỗi.
- Không nối field của `PublicProfileDto` — **đó là ranh giới chống rò rỉ dữ liệu user.**

2 Cấu trúc thư mục & đặt tên

docs/rules/folder.convention.md

2.1 Mười quy tắc **BẮT BUỘC**

- 1 **Mọi business logic mới thuộc một domain cụ thể** → nằm trong `src/features/<feature>/`, không rải ở `common/` hay `utils/`.
- 2 **Code chỉ được đưa vào `common/` khi được dùng bởi từ 2 feature trở lên.**
Một feature dùng → để trong feature đó.
- 3 **`config/` chỉ chứa khởi tạo kết nối/cấu hình hạ tầng**
(DB, Redis, logger, env, Swagger). Không viết business logic.
- 4 **Feature mới tối thiểu phải có**
`routes.ts`, `controller.ts`, `service.ts`, `schema.ts`. `model.ts` + `repository.ts` chỉ tạo nếu feature có schema Mongo riêng.
- 5 **Tên file: `<feature>.<layer>.ts`**
, toàn chữ thường, phân cách bằng dấu chấm. Đúng: `category.controller.ts`. Sai: `CategoryController.ts`, `category_controller.ts`.
- 6 **Middleware dùng chung nhiều feature** → `src/middlewares/`; **middleware riêng một feature** → **trong chính feature đó**
, tên `<feature>.middleware.ts`.
- 7 **Job vẫn nằm trong `src/jobs/`**
, nhưng import service từ feature để tái dùng logic — không viết logic mới trong `jobs/`.
- 8 **Test**
ở `tests/unit/` (hàm thuần) hoặc `tests/integration/` (qua HTTP). Tên: `<đối-tượng>.test.ts`.
- 9 **Không tạo thư mục cấp cao mới**
(ngang hàng `common`, `config`, `features`, `jobs`, `middlewares`, `sockets`, `tests`) nếu chưa được xác nhận. Thấy cần thì **dừng lại và hỏi**.
- 10 **`index.ts` trong `src/features/` chỉ để gom router**
— không import service/model của feature con vào đây.

2.2 Mười tám feature hiện có — không tự ý đổi tên hay gộp

`auth` · `category` · `chat` · `join-request` · `listing` · `location` · `membership` · `moderation` · `notification` · `org-unit` · `organization` · `report` · `review` · `role-grant` · `search` · `trust` · `upload` · `user`
`chain` và `platform-admin` **đã bị gỡ ở v2** — quyền hệ thống giờ nằm ở `role-grant`.

2.3 Checklist trước khi tạo file mới

- File này thuộc một feature cụ thể hay dùng chung nhiều feature?
- Đã đặt đúng thư mục chưa?
- Tên file có đúng format `<feature>.<layer>.ts` không?
- Nếu là middleware/job — đã xác định đúng phạm vi dùng chung hay riêng feature chưa?

- Có tạo thư mục cấp cao mới không? Nếu có → **dừng và hỏi**.

3 Hợp đồng API

docs/rules/api-contract.md

3.1 OpenAPI — code-first

- Spec **sinh từ Zod schema** qua `@asteasolutions/zod-to-openapi`. Không có file YAML/JSON viết tay.
- Component schema đăng ký ở `<feature>.schema.ts` bằng `registry.register(...)`.
- Endpoint đăng ký ở `<feature>.routes.ts` bằng `registry.registerPath(...)`, đặt ngay dưới khai báo route.
- **Mọi endpoint mới bắt buộc có `registerPath`**. Thiếu thì endpoint không xuất hiện ở `/docs` — bug này từng làm `/docs` trống hoàn toàn dù đã cài đủ Scalar + zod-to-openapi.
- Spec phục vụ tại `/openapi.json`, UI tại `/docs`.

3.2 Versioning

Mọi API có version trong đường dẫn: `/api/v1/...`

Không được thay đổi response field đã public

Bao gồm: đổi tên field · đổi kiểu dữ liệu · xoá field · đổi ý nghĩa/logic của field. Có breaking change thì **phát hành version mới** (`/api/v2/...`), và `v1` vẫn phải chạy cho tới khi có thông báo deprecation chính thức kèm thời gian chuyển tiếp.

3.3 Phân trang — format thống nhất

```
{ "success": true, "message": "Listings", "data": [],
  "meta": { "page": 1, "limit": 20, "total": 100, "totalPages": 5,
            "hasNextPage": true, "hasPrevPage": false } }
```

`meta` sinh bằng `buildPaginationMeta()`; tham số chuẩn hoá bằng `parsePagination()` (`src/common/utils/pagination.ts`).

Bắt đã biết

`total` phải đếm **cùng điều kiện** với `data` — hook soft-delete không tự áp cho `countDocuments` (xem quy tắc 10 ở phần 1).

3.4 Lỗi

Không `throw` tự do ra ngoài tầng API. Mọi lỗi đi qua `ApiError`.

```
{ "success": false, "message": "Listing not found",
  "details": [{ "path": "price", "message": "Expected number, received string" }] }
```

Field	Quy định
<code>success</code>	Luôn <code>false</code>
<code>message</code>	Thông điệp dễ hiểu, hiển thị được cho client/dev
<code>details</code>	Chỉ có với lỗi validation — danh sách <code>{ path, message }</code> từ Zod
<code>stack</code>	Chỉ ở dev , không bao giờ xuất hiện ở production

HTTP status nằm ở response status, **không lặp lại trong body**. Chưa có field `code` dạng `UPPER_SNAKE_CASE`; cần thì phải thêm vào `ApiError` và `errorResponseSchema` **cùng lúc**.

4 TypeScript

docs/rules/typesafty.convention.md

4.1 any REVIEW-ONLY

Không dùng `any`. Bắt buộc phải dùng (thư viện thiếu type) thì **phải có comment** nêu: lý do không dùng được type cụ thể hơn, và phạm vi sử dụng. Ưu tiên `unknown` khi cần kiểu "chưa xác định".

Luật này không có máy chặn

`.oxlinttrc.json` đang để `typescript/no-explicit-any: "off"` — phải soát ở code review.

4.2 Return type

Mọi `function` được `export` **bắt buộc khai báo return type tường minh**. Không để TS tự infer với public API. Lý do: phát hiện sớm breaking change ngoài ý muốn khi refactor.

```
// Sai
export function getUser(id: string) { return userRepository.findById(id) }
// Đúng
export function getUser(id: string): Promise<UserDto | null> { ... }
```

4.3 type hay interface

`type` object shape đơn giản, union type, kiểu không cần extend

`interface` model, DTO, cấu trúc có khả năng được extend về sau

4.4 DTO — ranh giới chống rò rỉ

Không trả trực tiếp database document ra ngoài API, đặc biệt ở endpoint không cần đăng nhập.

```
success(res, { data: user }) // SAI — lộ email, phone, lastLoginAt
success(res, { data: toPublicProfileDto(user) }) // ĐÚNG
```

Pattern: DTO + hàm map ở `<feature>.types.ts`, shape do Zod schema làm SoT (`type Dto = z.infer<typeof schema>`) để type, runtime và OpenAPI không lệch nhau. Mỗi luồng request/response có DTO riêng: `CreateListingDto`, `UpdateListingDto`, `ListingResponseDto`.

Hiện trạng ghi trong tài liệu — nợ kỹ thuật đã biết

`auth` và `GET /users/:id` đã dùng DTO. `GET/PATCH /users/me` và các endpoint `listing` vẫn trả document (đã lọc `password` qua `toJSON`) — cần DTO hoá khi có dịp đổi contract.

4.5 tsconfig

Bắt buộc `strict: true`. **Không được tắt** `strict` hay các flag con (`strictNullChecks`, `noImplicitAny` ...) chỉ để né lỗi biên dịch. Lỗi xử lý tận gốc trong code.

5 Ghi log

docs/rules/logging-monitoring.md

5.1 Structured logging

Chữ ký winston, không phải pino

```
logger.info({ event: 'user_login', userId }, 'User login') // SAI – pino
logger.info('user_login', { userId })                     // ĐÚNG – winston
```

Đảo thứ tự thì winston lấy object làm message, in ra [object Object] và nuốt luôn chuỗi mô tả.

Field	Nguồn
message	Tên sự kiện / mô tả ngắn — tham số thứ nhất
meta	Object context — tham số thứ hai (userId, err, path ...)
level	debug info warn error
timestamp	Tự sinh bởi transport
service	Từ defaultMeta, không cần truyền tay

- Truyền Error qua meta ({ err }) — formatter đã serialize message + stack; JSON.stringify mặc định trả {} vì hai field đó là non-enumerable.
- Không dùng console.log / console.error trong code nghiệp vụ (trừ scripts/).
- Không nuốt lỗi:** catch {} rỗng hoặc handler unhandledRejection rỗng là vi phạm — đã từng làm server chết mà không để lại một dòng log nào.

5.2 Dữ liệu tuyệt đối không được log

Dù ở dạng raw hay đã encode:

- Mật khẩu (password)
- Token xác thực — access token, refresh token, session token
- Số điện thoại đầy đủ** — cần thì phải mask (090****123)
- Dữ liệu thanh toán — số thẻ, CVV, thông tin tài khoản ngân hàng

Nên đưa vào cơ chế **redaction/masking tự động ở tầng logger**, thay vì phụ thuộc hoàn toàn vào developer.

5.3 Request trace — CHƯA TRIỂN KHAI

Mục tiêu, không phải hiện trạng

app.ts mới có access log (METHOD path status {ms}), **chưa có requestId**. Đừng viết code giả định requestId đã tồn tại.

Khi làm: mỗi request được gán requestId duy nhất (sinh mới, hoặc kế thừa từ x-request-id), truyền xuyên suốt HTTP → Service → Database → Queue, xuất hiện trong **mọi** log entry của request đó. Có message queue thì requestId phải dính trong metadata của message.

6 Multi-tenant — ranh giới bảo mật

docs/rules/multi-tenant.convention.md · 330 dòng · file nghiêm nhất

Một query quên filter không phải bug hiệu năng — nó là rò rỉ dữ liệu giữa hai khách hàng. Vì phạm \$0 và \$7 được coi là **lỗi bảo mật**, chặn tại code review, không merge rồi sửa sau.

v2 đã thay đổi bốn điều — phần này ghi đề mọi chỗ mâu thuẫn còn lại trong file

Tài liệu nói	v2 thực tế
chainReadable, đọc xuyên org trong chain	Chain đã bị gỡ. Thay bằng dualAxis — trực org và trực danh mục (reach: marketplace)
ORG_ROLES trên users	Tách đôi: memberships.role = thân phận, role_grants = quyền hạn
Nhánh /platform-admin/* + JWT type	Đã xoá. master là một role_grant scope system; JWT chỉ còn sub
Org lấy từ subdomain → orgSlug body → JWT	Header X-Org-Id → đối chiếu memberships ngay lúc đó

Ba điều không đổi: filter tenant nằm ở tenantPlugin chứ không ở repository · không có scope thì ném lỗi, không bao giờ query toàn DB · runUnscoped('lý do', ...) là lối thoát DUY NHẤT và phải grep ra được.

6.1 Sáu bất biến — vi phạm là reject ngay

- Mọi collection nghiệp vụ đều gắn tenantPlugin.**
Không có collection nào "tạm thời chưa cần tenant". 16 ngoại lệ đã duyệt — xem 6.3.
- Không tự viết filter organizationId trong repository/service.**
Scope đến từ context. Tự viết nghĩa là có hai nguồn sự thật, và cái viết tay sẽ sai trước.
- Ghi luôn rơi về ownOrgId.**
Không route nào, không role nào, không cờ nào mở rộng được phạm vi ghi. Bản ghi trực công khai (organizationId: null) là read-only với mọi org khác.
- Không có scope thì ném lỗi, không bao giờ query không filter.**
Fail-closed là mặc định. Thấy mình muốn thêm fallback "cho tiện" → dừng lại.
- organizationId do client gửi lên luôn bị bỏ qua.**
Nó chỉ đến từ token đã verify hoặc subdomain — không bao giờ từ body/query/params.
- Mọi index trên collection có tenant lấy organizationId làm khoá đầu tiên.**
Ngoại lệ duy nhất là TTL index (Mongo không cho compound).

6.2 Thêm collection mới

Gắn plugin trước, khai index sau

```
reportSchema.plugin(tenantPlugin) // PHẢI đứng trước
reportSchema.index({ organizationId: 1, createdAt: -1 })
```

Đảo thứ tự thì organizationId chưa tồn tại lúc khai index → **index sai âm thầm.**

Quyết định dualAxis — khai theo schema, không theo route

Câu hỏi: bản ghi này có sống được ngoài mọi tổ chức không?

Trả lời	Khai báo	Hiện có
Có	<code>plugin(tenantPlugin, { dualAxis: true })</code>	<code>Listing</code>
Không	<code>plugin(tenantPlugin)</code>	<code>Notification</code> , <code>OrgUnit</code> , và mặc định cho mọi collection mới

Bật lên đối hai thứ cùng lúc, phải nêu lý do nghiệp vụ trong PR: `organizationId` thành nullable, và khoá định tuyến không còn là `organizationId` mà là **reach** — `marketplace` đi về người phụ trách (danh mục × tỉnh), `members` / `group_open` đi về chính tổ chức.

Chỗ hay nhầm nhất

`organizationId` vẫn được ghi trên tin công khai của một thành viên, nhưng nó chỉ để **hiển thị nguồn gốc**, không quyết định ai duyệt. Đừng bật `dualAxis` chỉ vì "cần đọc rộng hơn một chút" — đọc rộng có chủ đích thì bọc `runUnscoped`, nó để lại vết grep được; `dualAxis` nối scope cho **mọi** truy vấn của collection.

6.3 Mười sáu collection không gắn plugin — và bù bằng gì

Collection	Vì sao không gắn	Bù bằng
User	Tài khoản là toàn cục ở v2	Quan hệ với org nằm ở <code>Membership</code> ; <code>User</code> không có cột <code>organizationId</code> lẫn <code>role</code>
Membership	Chính nó trả lời "request đang ở org nào" → phải đọc được TRƯỚC khi scope tồn tại	Mọi method repository nhận <code>organizationId</code> tường minh, ép bằng kiểu
RoleGrant	Nguồn của phân quyền, cũng phải đọc trước scope	<code>roleGrantService.grantsOf(userId)</code> → <code>authz/policy.ts</code>
JoinRequest	Người gửi theo định nghĩa chưa thuộc org đích	Org đích đi trong body (mã nhóm); hàng đợi lọc tường minh
Invite	Người được mời chưa thuộc org đích — lượt đọc lúc bấm link không có scope	Repository nhận <code>organizationId</code> tường minh; đường thu hồi so bằng tay trước khi ghi
Notification	Hộp thư thuộc người nhận , không thuộc tổ chức; người thuộc hai org phải đọc được cả hai trong một hộp thư	Nhánh phát chung (<code>userId: null</code>) vẫn lọc tường minh qua <code>broadcastFilter</code>
UserTrust	Uy tín thuộc tài khoản , mà tài khoản ở v2 là toàn cục	Bậc chỉ đổi qua một đường: <code>PATCH /moderation/listings/:id</code>
Favorite	Gắn plugin thì người chưa vào org nào — phần lớn người mua — không lưu nổi tin nào	Bảng chỉ giữ (<code>userId</code> , <code>listingId</code>); nội dung tin vẫn đọc qua <code>Listing</code> nơi plugin còn nguyên
Organization	Chính nó là tenant	Chỉ truy cập qua repository của feature đó
Category	Dùng chung toàn hệ thống	Không có dữ liệu riêng của khách hàng
FieldDefinition	Từ điển field của template tin đăng	Ghi chỉ qua <code>scripts/seed-templates.ts</code>
CategoryTemplate	Gắn với <code>Category</code> , mà <code>Category</code> đã ngoài tenant	API chỉ mở đường ĐỌC
BannedPhrase	Luật áp toàn nền tảng , một org không được tự nối cho khu vực mình	Ghi chỉ qua API master-only <code>/banned-phrases</code>
ListingProduct	Catalog gói tin — bảng giá áp toàn nền tảng	Ghi chỉ qua API master-only <code>/listing-products</code>
Wallet	Ví Xu thuộc tài khoản; gắn plugin thì đổi org lại thấy số dư khác	Số dư chỉ đổi qua <code>walletService.apply()</code> trong transaction cùng dòng sổ cái
XuTransaction	Sổ cái của ví, đi cùng <code>Wallet</code>	Append-only, <code>idempotencyKey</code> unique; không có đường sửa/xoá

Muốn thêm một collection vào danh sách này → dừng lại và hỏi, không tự quyết.

6.4 Repository & Service

```
findActive() { return Report.find({ status: 'open' }) } // ĐÚNG
findActive(orgId) { return Report.find({ status: 'open', organizationId: orgId }) } // SAI
```

Cần gì	Dùng	Khi không có scope
Bắt buộc phải có scope	<code>requireScope('tên thao tác')</code>	Ném <code>TenantScopeMissingError</code> (400)
Có thì dùng, không có cũng được	<code>currentScope()</code>	Trả <code>undefined</code>

Service của collection không gắn plugin phải tự `requireScope()`. **Đừng nhận `organizationId` từ controller** — controller lấy ở đâu ra thì lại là một câu hỏi nữa.

Không populate xuyên tenant

```
Listing.find().populate('seller') // SAI – User không có plugin
```

Cần hiển thị dữ liệu của entity thuộc org khác → **denormalize snapshot** vào chính bản ghi lúc tạo (`Listing.posterName / posterContact` là mẫu). Nhớ thêm: `populate` vào model **chưa tồn tại** ném `MissingSchemaError` lúc runtime, TS không bắt được.

6.5 Index

Loại	Prefix <code>organizationId</code> ?	Ghi chú
Index thường	bắt buộc	Thiếu prefix thì org lớn nhất làm chậm mọi org còn lại
Index của trục danh mục (<code>dualAxis</code>)	prefix là <code>reach</code>	Bản ghi ở trục này có <code>organizationId: null</code> . Đủ hai họ index
<code>2dsphere</code>	được	<code>\$near</code> vẫn dùng được index
TTL	không thể	Mongo từ chối compound TTL. Là tiến trình dọn nền, không nằm trên đường query
Text index	CẤM DỪNG	Prefix bắt buộc equality, mà scope đọc mặc định là <code>\$in</code> nhiều org → vỡ. Full-text đi đường Atlas Search

Unique + soft delete: luôn kèm `partialFilterExpression: { deletedAt: null }`, nếu không một bản ghi đã xóa sẽ giữ chỗ giá trị unique vĩnh viễn.

6.6 Route & middleware

```
resolveTenant → authenticate → (requireOrg* / require*Moderator) → validate → controller
```

- `resolveTenant` mount ở `app.ts` cho cả `API_PREFIX`, **không mount lại ở feature**.
- Token không mang org nữa (payload chỉ có `sub`). Câu hỏi "người này có thuộc org đang resolve không" do chính `resolveTenant` đối chiếu `memberships` tại thời điểm đó — nên không còn cách nào cầm token cũ đi vào org khác.
- `requireCategoryModerator` / `requireMasterPublicAxis` nằm trong feature `moderation`; chỉ `requireOrg`, `requireMembership`, `requireOrgModerator`, `requireOrgAdmin` mới lên `src/middlewares/`.

```
runWithTenant(scope, next) // ĐÚNG
runWithTenant(scope, () => {}); next() // SAI – context rỗng ngay lập tức
```

Không để route "xem xuyên org". Dữ liệu `dualAxis` đã có mặt trong route đọc thông thường; hai đường đọc cho cùng một dữ liệu thì sớm muộn sẽ phân quyền lệch nhau. Ngoại lệ đã duyệt: `GET /notifications?scope=managed` — nó **đổi câu hỏi** ("tôi gửi được tới đâu"), không nói phạm vi.

6.7 Chạy code ngoài request — `runUnscoped`

1 reason viết cụ thể

, không viết `'script'`. `grep -rn "runUnscoped"` là danh sách kiểm toán mọi lỗi đi xuyên tenant — nó chỉ có giá trị khi lý do đọc được.

- 2 Trong `runUnscoped`, plugin **không** gán `organizationId` hộ. Document thiếu field này bị từ chối bằng `CrossTenantWriteError`.

3 **Query của Mongoose là lazy**

— `.exec()` phải nằm TRONG callback:

```
await runUnscoped('audit', () => Report.find()) // SAI
await runUnscoped('audit', () => Report.find().exec()) // ĐÚNG
```

- 4 Thêm một call site `runUnscoped` mới là thay đổi đáng review kỹ. Nằm trong luồng request thì **gần như chắc chắn đang làm sai** — hỏi trước.

6.8 Tám anti-pattern cấm tuyệt đối

Anti-pattern	Vì sao	Thay bằng
Cờ bypass kiểu <code>crossTenant: true</code>	Có một đường thoát là có mọi đường thoát	<code>runUnscoped('lý do', ...)</code> — grep được
<code>estimatedDocumentCount()</code>	Đọc metadata cả collection, không nhận filter	<code>countDocuments()</code> — plugin đã chặn cứng
<code>pre('save')</code> để gán <code>organizationId</code>	Mongoose chạy validation TRƯỚC pre-save	<code>pre('validate')</code> — plugin đã làm sẵn
<code>organizationId</code> mutable	Cho phép chuyển bản ghi sang org khác	<code>immutable: true</code> — plugin đã set
<code>populate</code> sang collection không có plugin	Lách được cách ly	Snapshot field
Filter <code>organizationId</code> viết tay trong service	Hai nguồn sự thật	Để plugin làm
Bỏ <code>partialFilterExpression</code> ở unique index	Bản ghi đã xóa giữ chỗ vĩnh viễn	Luôn kèm <code>{ deletedAt: null }</code>
Text index trên collection có tenant	Vỡ với scope <code>\$in</code> nhiều org	Atlas Search

6.9 Test bắt buộc trước khi merge

Thêm collection có tenant, hoặc đổi bất cứ thứ gì trong `src/common/tenant/`, thì **không nhận "code review thấy ổn" làm bằng chứng**:

- Org A không đọc được bản ghi của org B → **404**, không phải 403 (403 là lộ sự tồn tại)
- Org A không sửa/xóa được bản ghi của org B
- Không có scope → throw, KHÔNG trả về toàn bộ document
- `organizationId` client gửi lên bị bỏ qua (cả `create` lẫn `insertMany`)
- Nếu bật `dualAxis`: bản ghi `organizationId: null` đọc được từ mọi org, nhưng **ghi thì không**

Mẫu: `tests/unit/tenantPlugin.test.ts` và `tests/integration/tenant-isolation.test.ts`. Integration test dựng luồng đăng ký Organization phải dùng `MongoMemoryReplSet` — transaction không chạy trên standalone.

6.10 Checklist trước khi mở PR

- Collection mới đã `plugin(tenantPlugin)`, `dualAxis` để mặc định tắt trừ khi PR nêu rõ lý do

- Mọi index mới có `organizationId` đứng đầu (trừ TTL)
- Unique index kèm `partialFilterExpression: { deletedAt: null }`
- Repository không có chuỗi `organizationId` viết tay
- Không có `populate` sang collection thiếu plugin
- `runUnscoped` mới (nếu có) đã được nêu trong PR description kèm lý do
- Có test cách ly theo 6.9
- `npm run lint` · `npm run typecheck` · `npm test` đều pass
- Đổi schema → ghi rõ trong PR có cần chạy `npm run migrate:tenant` không (xem 9.2)

7 ADR — Hệ Xu và ví

docs/rules/xu-wallet.decision.md

Trạng thái: giai đoạn 1 (ví) **đã xây**; giai đoạn 2 (nap) **đã dựng rồi gỡ bỏ có chủ ý** (23/08/2026). Phí đăng tin vẫn = 0. Tài liệu tồn tại để ngày bật phí không cần thiết kể lại gì, chỉ thực thi.

7.1 Bốn bất biến của sổ cái

1 Sổ cái append-only

— sai thì ghi dòng điều chỉnh ngược dấu, không sửa/xoá.

2 Mọi biến động đi qua đúng MỘT hàm

(`walletService.apply`), có `idempotencyKey` .

3 "Trừ Xu + tạo tin" chạy trong một transaction Mongo.

Hạ tầng bắt buộc là

replica set

(Atlas mặc định có; dev local cần `--replSet`).

4 Xu một chiều

— nạp vào được, không quy đổi ngược ra tiền. Giữ nền tảng ngoài phạm vi giấy phép trung gian thanh toán; phải ghi rõ trong điều khoản sử dụng.

Cấm `user.xu` cộng trừ tại chỗ. Hai collection `wallets` + `xu_transactions` , cả hai tenant-exempt. `balance` chỉ là **cache** của sổ.

7.2 Chính sách đã quyết

Tình huống	Quyết định	Vì sao
Tin bị từ chối khi duyệt	Hoàn Xu	Trả phí để <i>được đăng</i> , không phải để xếp hàng
Tin bị gỡ sau khi lên bảng	Không hoàn	Đã hưởng dịch vụ; cùng họ với trừ uy tín
Cổng nội dung chặn từ cửa	Không trừ từ đầu	Chỉ trừ khi tin vào <code>PENDING / ACTIVE</code>
Xu có hạn sử dụng	Không	Xu hết hạn là nguồn khiếu nại số một
Uy tín cao giảm phí	Để ngỏ	Hook sẵn trong <code>postingFee</code>
Giá theo danh mục	Để ngỏ	Quyết bằng số liệu <code>posting-stats</code>
Miễn phí N tin/tháng sau khi bật	Nên có	Chuyển tiếp mềm cho người dùng vắng lai
Tặng Xu khai trương	Có (<code>promo_grant</code>)	Ngày bật phí phải là ngày nhận quà

7.3 Sáu bài học từ bản nạp tiền đã gỡ

Giữ lại để khỏi phải phát hiện lần nữa khi dựng lại:

1 Tiền tố mã đối soát **KHÔNG** được là `NAP`

— bỏ dấu xong thì chữ "nap" người Việt luôn gõ trong nội dung chuyển khoản cũng thành `NAP` . Dùng tên ứng dụng (`GHIM`) và trả về

danh sách ứng viên

để phép tra DB làm trọng tài.

2 VietQR là hai thứ khác nhau

: `img.vietqr.io` sinh ảnh QR miễn phí không cần đăng ký; `api.vietqr.org` (VietQR JSC + MB) mới là dịch vụ đổi soát — có sandbox `dev.vietqr.org` test được webhook không cần một đồng tiền thật.

3 Webhook của họ bắt merchant tự dựng

cả endpoint `token_generate`

(Basic → Bearer), không phải chỉ kiểm một API key như SePay.

4 `transType: D` là tiền RA

— cộng Xu cho nó là biểu không tiền cho người vừa rút.

5 Giao dịch đã xử lý rồi mà trả `error: true` là tự tạo

vòng lặp retry vô tận

của họ.

6 Cộng Xu theo số tiền THẬT nhận được

, không theo số tiền đơn hèn.

7.4 Gói tin

Catalog sống trong DB (`ListingProduct`) — master xuất bản/sửa/ngừng bán qua `/listing-products` , không cần deploy. `GET /listings/products` công khai **chỉ** trả gói đang `enabled` .

Luật cứng của catalog

- Mở bán **phải có giá**.
- code **bất biến** — số cái tham chiếu bằng nó. Đổi bản chất gói = tạo gói mới, ngừng bán gói cũ.
- Mua rồi thì số cái **snapshot điều khoản**; sửa gói không viết lại lịch sử.

Gói	Hiệu ứng	Cơ chế
bump — Đẩy tin	<code>rankAt = now</code>	<code>rankAt</code> là khoá sắp xếp MỚI của mọi bảng tin. Không sửa <code>createdAt</code> — đó là lịch sử
featured_3d/7d	<code>featuredUntil = now + N ngày</code>	FE suy badge từ <code>> now</code> ; field đã có trong model + DTO
extend_30d — Gia hạn	<code>expiresAt += 30 ngày</code>	TTL 30 ngày sẵn có
Tình huống	Quyết định	
Điều kiện mua	Tin <code>ACTIVE</code> + của chính mình — gói tin là hiển thị , không phải đường tắt qua duyệt	
Chống spam đẩy	Cooldown 24h/tin — chặn cả nhà giàu chiếm bảng	
Tin nổi bật bị sửa → về <code>PENDING</code>	Đồng hồ chạy tiếp ; UI cảnh báo trước	
Tin nổi bật bị gỡ vì vi phạm	Không hoàn Xu	
Tin hết hạn khi đang nổi bật	Hết là hết; muốn thì mua gia hạn, không tự trừ tiền	
Đẩy tin có chen hàng DUYỆT không	KHÔNG — hàng đợi giữ <code>createdAt</code> , FIFO theo lúc đăng thật	

Vận hành khi deploy

Chạy `npm run migrate:rank-at` **ngay sau deploy** — backfill `rankAt = createdAt` cho tin cũ + đồng bộ họ index. Chưa chạy thì tin cũ **chìm xuống đáy bảng** (BSON xếp field vắng mặt nhỏ hơn mọi Date).

7.5 Checklist ngày bật phí

- posting-stats có tối thiểu ~2 chu kỳ 30 ngày dữ liệu để định giá
- Giai đoạn 1 + 2 đã chạy ổn (ví + nạp), có người nạp thật
- Điều khoản sử dụng có mục Xu (một chiều, không hoàn tiền mặt, luật hoàn Xu)
- Test canary trong `listingPricing.test.ts` được sửa **có ý thức** cùng lượt đổi giá

8 Luật nào có máy chặn, luật nào không

Đối chiếu `.oxlintrc.json` · `.husky/pre-commit` · `package.json` – không nằm trong file luật nào

Tài liệu gốc đều kết bằng câu "các quy tắc trên là bắt buộc". Nhưng **bắt buộc** và **được cưỡng chế** là hai chuyện. Bảng dưới là hiện trạng đo được.

8.1 Ba cơ chế duy nhất đang chạy

Cơ chế	Chạy khi nào	Bắt được gì
<code>.husky/pre-commit</code> → <code>lint-staged</code>	Mỗi lần <code>git commit</code>	oxlint + Prettier trên file đã stage
<code>npm run typecheck</code>	Tay / CI	Lỗi kiểu — <code>strict</code> đang bật
<code>npm test</code> (Vitest)	Tay / CI	Gồm <code>tenantPlugin.test.ts</code> + <code>tenant-isolation.test.ts</code> — đây là chốt thật của phần 6

8.2 oxlint đang bật những gì

```
plugins:    typescript, oxc, unicorn
categories: correctness = error, suspicious = warn
rules:      no-console: off           ← quy tắc 1.3 §9 KHÔNG được lint chặn
            typescript/no-explicit-any: off ← luật 4.1 KHÔNG được lint chặn
            no-unused-vars: warn
            no-underscore-dangle: warn (cho phép _id, __v)
```

8.3 Luật quan trọng nhưng chỉ sống nhờ code review

Luật	Ở phần	Vì sao không gate được
Không dùng <code>any</code>	4.1	<code>no-explicit-any</code> đang <code>off</code> trong config
Không <code>console.log</code>	1.3 §9	<code>no-console</code> đang <code>off</code>
Thứ tự tham số winston vs pino	5.1	Cả hai đều hợp lệ về kiểu — chỉ sai về ngữ nghĩa
Controller không chứa logic nghiệp vụ	1.3 §1	Là câu hỏi về ý định, không phải hình dạng AST
Code chỉ vào <code>common/</code> khi ≥ 2 feature dùng	2.1 §2	Quan hệ xuyên file
Return type tường minh cho hàm export	4.2	Rule tồn tại nhưng chưa bật
Không trả document thẳng ra API	4.4	Tài liệu tự nhận còn nợ ở <code>/users/me</code> và <code>listing</code>
Endpoint mới phải có <code>registerPath</code>	3.1	Thiếu thì <code>/docs</code> im lặng trống
Không log dữ liệu nhạy cảm	5.2	Tài liệu đề xuất redaction tự động ở tầng logger, chưa làm

Kết luận thực dụng

Phần 6 (multi-tenant) là phần duy nhất có **cả ba lớp**: plugin chặn ở runtime, test chặn ở CI, và luật chặn ở review. Mọi phần còn lại phụ thuộc gần như hoàn toàn vào người đọc code.

9 Mâu thuẫn đã phát hiện

Quan sát khi đối chiếu, không có trong tài liệu gốc

9.1 multi-tenant.convention.md §5 mâu thuẫn với chính banner đầu file

§5 "Auth & Token" nói	Chỗ khác trong cùng file nói
"JWT payload bắt buộc có type ('user' 'platform_admin')"	"Nhánh /platform-admin/* + JWT type đã xoá . JWT chỉ còn sub " — banner đầu file
"Token user bắt buộc mang organizationId "	"Token KHÔNG mang org nữa (payload chỉ có sub)" — §4.1
"email chỉ unique trong phạm vi (organizationId, email) "	"User là toàn cục ở v2, không có cột organizationId " — §1.3

Cách xử lý đã ghi sẵn trong tài liệu

Banner nói rõ: "đọc mục này trước, nó **ghi đè** mọi chỗ mâu thuẫn trong phần còn lại của file". Vậy §5 là phần **chưa được dọn** sau đợt v2, không phải luật đang hiệu lực. Ba điều còn đúng trong §5: không rút thông tin tenant từ body/query để phân quyền · orgId trong body chỉ dùng để *resolve org*, không để cấp quyền · nơi JWT_EXPIRES_IN phải có lý do, vì suspend org dựa vào token ngắn + check status live.

9.2 Hai điểm nhỏ hơn

- §0 liệt kê 16 collection miễn trừ, §1.3 giải thích đủ 16 — hai danh sách khớp nhau. Không có lệch; ghi lại để khỏi phải kiểm lần nữa.
- migrate:tenant trong checklist PR (6.10) không có trong package.json. Các script migrate thật mang tên khác: migrate:v2, migrate:trust, migrate:rank-at, migrate:listing-reach, migrate:ward-axis, migrate:listing-expiry, migrate:org-admin ... Cần xác minh ý định trước khi làm theo dòng đó.

9.3 Tài liệu liên quan, không phải luật

Bây file trong docs/architecture/ là **thiết kế và lý do**, không phải luật ràng buộc — nhưng v2-org-permission.plan.md được tài liệu luật trở tới nhiều lần như trạng thái đích của v2, nên đọc nó trước khi chạm hệ phân quyền. rules-link.md chỉ chứa link tới một bản giải thích bậc uy tín cho người không rành kỹ thuật; nguồn sự thật vẫn là src/features/trust/ và src/features/listing/listing.quota.ts.